User Story	To-Do	In Progress	Done
As a student, I want to see my live GPS location and get a simple walking path to my class.			Live GPS Integration: Implement the browser Geolocation API to place a "You Are Here" marker on the map to help students orient themselves relative to campus landmarks. (4) - Juan One-Tap Navigation: Add a "Get Directions" button to every class pin that draws a simple walking route from the student's current GPS position to the building entrance. (8) - Jon UI Focus Mode: Develop a "Recenter Map" button that appears only when the user is off-screen, keeping the interface uncluttered. (4) - Alexis
As a student, I want to see what the building looks like so I can recognize it when I arrive.			Room Number Emphasis: Redesign the InfoWindow to make the "Room Number" the most prominent visual element once a pin is clicked. (4) - Jon Building Image Integration: Finalize the image engine to ensure building photos from the database appear in the map InfoWindow. (6) - Juan Dark Theme: Develop a system-wide dark mode. This includes creating a custom Google Maps JSON style for dark tiles, refactor style.css to use a complete variable-based theme system, and implementing a toggle in script.js that persists the user's preference in localStorage. (8)
As a student, I want my search results to clearly distinguish between physical, online, and TBD classes so I only attempt to map reachable locations.			Smart Autocomplete: Implement a search suggestion endpoint in the Go backend to help students find courses with fewer keystrokes. (8) Status-Based Blocking & Notification: Modify the search dropdown logic to identify TBD or Cancelled classes. If a student tries to add one, block the addition and trigger a UI notification/toast explaining the status. (8) Online Instruction Handling: Clearly badge "Online/Remote" classes in the search results to distinguish them from TBD classes. (4)
As a student, I want the app to automatically display the most relevant academic terms so I can look up classes for the current or next quarter without encountering outdated info.			Available Terms API: Implement a Go endpoint /api/terms that queries the database for DISTINCT term and returns a sorted list to the frontend. (3) Dynamic Dropdown Injection: Update script.js to fetch available terms on page load and dynamically populate the #term-select element, removing the need for manual HTML edits. (4) Date-Driven Scraper Automation: Refactor scraper.py to calculate upcoming UCSC term codes (STRM) based on the system clock rather than a hardcoded list. (6) - Jon Intelligent Term Defaulting: Implement logic to automatically pre-select the most "current" term in the dropdown based on the date, so the user doesn't have to click anything. (2)